Data and Text Matching

Acknowledgement: Material in these slides taken from open sources and from ‘Principles of Data Integration Book by Anhai…

P2P evaluation of Quiz --

Assume that we have a single table that has millions of records of Indian electoral. The table may have duplicate records. We need to flag these duplicates. What are the technical challenges you see here?

Why Matching Matters

- A relational data is a collection of entities and/or can be extracted from unstructured data.
  - ➢ Challenges in handling -- Data String Variant, Typos, missing values, abbreviations, inconsistent formats, etc
- String matching: Compare attribute values (e.g., names, addresses)

  Record matching: Compare entire records with multiple fields

  String similarity is a sub-problem of entity resolution
- **Find strings that refer to same real-world entities**
  - "Venketesh Vishwanathan" vs "Venktesh Vishvanathan"
  - Harpreet Kaur vs Hary Singh Kaur,
  - Debjoot Vohara vs Deb Vhora
  - Daniel Walker Herbert Smith vs. Dan W. Smith
  - "Room-108, Boys Students Residence, IIITD, Delhi 20" vs

  "Room# 108, Boys Hostel, IIIT-D, Okhla Phase 3, New Delhi 110020"

Operation	Worked transformation	Failure to guard against
Typed parsing	12/04/1980 → 1980-04-12 only with a known day/month locale	Ambiguous dates remain unresolved
Contextual dictionary	M.G. Rd → MG Road → Mahatma Gandhi Road in this locale	MG may mean something else elsewhere
Attribute extraction	“ACME X2 128GB, pack of 2” → model=X2, capacity=128, pack=2	Missing and inferred values differ
Canonical units	0.5 g → 500 mg	500 mg and 500 mL are incompatible

Why Matching Matters

- Data integration from multiple sources
  - Entity resolution in business, healthcare, e-commerce
  - Handling duplicates, inconsistencies, and variations in text
  - Improves data quality and downstream analytics

- Matching Methods:
  - Deterministic: Rule-based, exact/approximate string comparison
  - Probabilistic: Statistical models (e.g., Fellegi-Sunter)
  - Machine Learning: Feature-based models
  - Deep Learning: Representation learning with neural networks
  - Hybrid and active learning approaches

- Examples:
  - Customer profile linking across platforms
  - Product catalog integration (Amazon-Flipkart datasets)
  - Deduplication of patient records in healthcare
  - Fraud detection (matching suspicious entities)

The representation determines the matching algorithm

Task	Example	Algorithm and cost
Exact equality	Same validated source identifier	Hash join: expected O(|A|+|B|+output) overall
Aligned characters	AB12 versus AB13	Hamming = 1, O(L), equal lengths required
Inserted / deleted text	Sumit versus Somith	Levenshtein dynamic programming, O(mn)
Reordered tokens	“Rao Martha” versus “Martha Rao”	Set Jaccard or weighted cosine
Semantic variants	“wireless earbuds” versus “Bluetooth earphones”	Learned representation, then identity checks

m and n denote string lengths. L denotes a common length. Output size remains unavoidable.

Methods for
Data String Matching

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - **Sequence-based:** edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering.

Problem Description

Mukesh K Mohania, PhD 0091-88007-61686 CSE Department, IIITD, Delhi 20
M Kumar Mohaniya 8800761668 Dept of Comp Sc&Engg, IIIT Okhla Phase 3, ND 11020

- Given two sets of strings X and Y
  - Find all pairs x∈X and y∈Y that refer to the same real-world entity
  - We refer to (x,y) as a match
  - Example

Set X	Set Y	Matches
x_1 = Dave Smith	y_1 = David D. Smith	(x_1, y_1)
x_2 = Joe Wilson	y_2 = Daniel W. Smith	(x_3, y_2)
x_3 = Dan Smith		
(a)	(b)	(c)

- Two major challenges: accuracy & scalability

(Similarity Measures) Matching Methods

Exact Matching (simplest form)

- Two strings are considered identical if they match exactly

- Example:
  - 'Mukesh K Mohania' vs 'Mukesh K Mohania' → Match
  - 'Mukesh K Mohania' vs 'Mukesh Mohania' → No Match

- Use Case: Matching IDs, standard codes, exact product SKUs

Normalization based matching

- Pre-processing step: normalize strings before comparison

- Operations: case folding, whitespace removal, punctuation stripping

- Example:
- 'Mukesh Kumar Mohania' vs 'Mukesh kumar Mohaniya'
  → Normalize → 'mukeshkumarmohania' vs 'mukeshkumarmohaniya' → Match

- Use Case: Cleaning text before exact or fuzzy matching

Matching Methods – contd…

Rule based matching

- Uses domain-specific rules for equivalence
- Example: Abbreviation Expansion
  - **Rule**: 'St.' ≈ 'Street', 'Ave' ≈ 'Avenue'
  - '123 Mahatma Gandhi Rd' vs '#123 Mahatma Gandhi Road' → Match

- Example: Synonym Mapping
  - **Rule**: ‘Bombay' ≈ ‘Mumbai'
  - ‘BG Taxi' vs ‘Blue Green Taxi' → Match

Similarity measures

- Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith- Waterman, Jaro, Jaro-Winkler
- Set-based: overlap, Jaccard, TF/IDF
- Hybrid: generalized Jaccard, soft TF/IDF, Monge-Elkan
- Phonetic: Soundex

Scaling up string matching

- Inverted index, size filtering, prefix filtering, position filtering, bound filtering

P2P evaluation of Quiz --

Assume that we have a single table that has multi millions of records of Indian electoral. The table may have duplicate records. We need to flag these duplicates. What are the technical challenges you see here?

Accuracy Challenges

- Solutions:
  1. Use a similarity measure s(x,y) ∈ [0,1]
     - The higher s(x,y), the more likely that x and y match
     - Declare x and y matched if s(x,y) ≥ t
  2. Distance measure/cost measure have also been used
     - smaller values means higher similarities

Scalability Challenges

Applying s(x,y) to all pairs is impractical

- Quadratic in size of data

Solution: apply s(x,y) to only most promising pairs, using a method

FindCands

For each string x ∈ X

use method *FindCands* to find a candidate set Z □ Y

for each string y ∈ Z

if s(x,y) ≥ t then return (x,y) as a matched pair

We discuss ways to implement *FindCands* later

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - **Sequence-based:** edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering.

Edit Distance

- Also known as Levenshtein distance
- Use Case: Handling typos and misspellings
- d(x,y) computes minimal cost of transforming x into y, using a sequence of operators, each with cost 1
  - Delete a character
  - Insert a character
  - Substitute a character with another
- Example: x = David Smiths, y = Davidd Simth, d(x,y) = 4, using following sequence
    - Inserting a character d (after David)
    - Substituting m by i
    - Substituting i by m
    - Deleting the last character of x, which is s

Edit Distance

- Models common editing mistakes
  - Inserting an extra character, swapping two characters, etc.
  - So smaller edit distance higher similarity
- Can be converted into a similarity measure
  - s(x,y) = 1 - d(x,y) / [max(length(x), length(y))]
  - Example
    - s(David Smiths, Davidd Simth) = 1 – 4 / max(12, 12) = 0.67

What would be the s(x,y), where x= Venketesh Vishvanatan and y= Venktesh Vishwanathan ?

Computing Edit Distance using Dynamic Programming

- Define x = x1x2… xn, y = y1y2… ym
  - d(i,j) = edit distance between x1x2… xi and y1y2… yj, the ith and jth prefixes of x and y
- Recurrence equations

$$d(i,j) = \min \begin{cases} d(i-1,j-1) & \text{if } x_i = y_j \quad \text{// copy} \\ d(i-1,j-1) + 1 & \text{if } x_i <> y_j \quad \text{// substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$d(i,j) = \min \begin{cases} d(i-1,j-1) + c(x_i,y_j) & \text{// copy or substitute} \\ d(i-1,j) + 1 & \text{// delete } x_i \\ d(i,j-1) + 1 & \text{// insert } y_j \end{cases}$$

$$c(x_i,y_j) = \begin{cases} 0 & \text{if } x_i = y_j, \\ 1 & \text{otherwise} \end{cases}$$

Example

- x = dva, y = dave

		y0	y1	y2	y3	y4
			d	**a**	**v**	**e**
x0		0	1	2	3	4
x1	**d**	1	0	1		
x2	**v**	2				
x3	**a**	3				

		y0	y1	y2	y3	y4
			d	**a**	**v**	**e**
x0		0	1	2	3	4
x1	**d**	1	0	1	2	3
x2	**v**	2	1	1	1	2
x3	**a**	3	2	1	2	2

x = d – v a
| | | |
y = d a v e

substitute a with e
insert a (after d)

The Jaro Measure

- Mainly for comparing short strings, e.g., first/last names

- To compute jaro(x,y)
  - find common characters **x**i and **y**j such that
    - **x**i = **y**j and |i-j| $\leq$ min {|x|,|y|}/2,
  - intuitively, common characters are identical and positionally "close to each other"
  - Compute the matching window: A character in one string can match a character in the other if it lies within ± (Max (|x|,|y|)/2)-1 positions.
    - if a character of x does not match the character of y, then we have a transposition
  - return jaro(x,y) = 1 / 3[c/|x| + c/|y| + (c – t/2)/c], where c is the number of common characters, and t is the number of transpositions

The Jaro Measure: Examples

$$jaro(x,y) = 1 / 3[c/|x| + c/|y| + (c - t/2)/c],$$

- x = jon, y = john
  - c = 3 because the common characters are j, o, and n
  - t = 0
  - jaro(x,y) = 1 / 3(3/3 + 3/4 + 3/3) = 0.917
  - contrast this to 0.75, the similarity score of x and y using edit distance
- x = jon, y = ojhn
  - c = 3 because the common characters are j, o, and n
  - t = 2
    - common char sequence in x is jon
    - common char sequence in y is ojn
  - jaro(x,y) = 0.81

The Jaro-Winkler Measure

jaro(x,y) = 1 / 3[c/|x| + c/|y| + (c – t/2)/c],

- Captures cases where x and y have a low Jaro score, but share a prefix, still likely to match
- Computed as
  - jaro-winkler(x,y) = (1 – PL*PW)*jaro(x,y) + PL*PW
  - PL = length of the longest common prefix
  - PW is a weight given to the prefix
- Exercise: Compute the edit distance between "Vishwanathan" and "Vishvnahtan" using Levenshtein, Jaro and Jaro-Winkler. Here PW=0.1

Edit Distance using Levenshtein: 4

- s1 = "Vishwanathan" (length 12) s2 = "Vishvnahtan" (length 11)
- **Match window**: ⌊max(12,11)/2⌋−1=5
  (A character in one string can match a character in the other if it lies within ±5 positions.)
- **Matched characters (in the order they are found)**
  - From s1 (matched): ['V','i','s','h','a','n','a','t','h','n'] (second last 'a' is not pairing up in s2)
  - From s2 (matched): ['V','i','s','h','n','a','h','t','a','n']

C=10, t=4, Jaro Score: 0.847, JaropWinkler: 0.9082

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - **Set-based:** overlap, Jaccard, Sorensen-Dice, TF/IDF
  - **Similarity-based:** Ratcliff-Obershelp similarity
  - **Phonetic:** Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering, bound filtering

Set-based Similarity Measures

- View strings as sets or multi-sets of tokens
- Use set-related properties to compute similarity scores
  - Find the similar tokens in both sets. More the number of common tokens, more is the similarity between the sets.
- Common methods to generate tokens
  - consider words delimited by space
    - possibly stem the words (depending on the application) [catch, catchy, catcher,..)
    - remove common stop words (e.g., the, and, of)
    - e.g., given “david smith”, generate tokens “david” and “smith”
  - consider n-grams, substrings of length n
    - e.g., “david smith”, the set of 3-grams are {##d, #da, dav, avi, ..., h##}
    - special character # is added to handle the start and end of string.

The Overlap Measure

- Let **B**x = set of tokens generated for string x
- Let **B**y = set of tokens generated for string y
- O(x,y) = |**B**x ∩ **B**y|
  - returns the number of common tokens
- E.g., x = dave, y = dav
  - **B**x = {#d, da, av, ve, e#}, **B**y = {#d, da, av, v#}
  - O(x,y) = 3

The Jaccard Measure

- J(x,y) = |Bx ∩ By|/|Bx ∪ By|
- E.g., x = dave, y = dav
  - Bx = {#d, da, av, ve, e#}, By = {#d, da, av, v#}
  - J(x,y) = 3/6
- Very commonly used in practice

The TF/IDF Measure: Motivation

- uses the TF/IDF notion commonly used in IR
  - two strings are similar if they share distinguishing terms
  - e.g., x = Apple Corporation, CA

    y = IBM Corporation, CA

    z = Apple Corporation
  - s(x,y) > s(x,z) using edit distance or Jaccard measure, so x is matched with y ➔ incorrect
  - TF/IDF measure can recognize that Apple is a distinguishing term, whereas Corporation and CA are far more common ➔ correctly match x with z

Term Frequencies and Inverse Document Frequencies

- Each string is converted into a bag of terms called a document
- Define term frequency tf(t,d) = number of times term t appears in document d
- Define inverse document frequency idf(t) = N / Nd, number of documents in collection divided by number of documents that contain t
  - A higher value of IDF means that the occurrence of the term is more distinguishing.
  - note: in practice, idf(t) is often defined as log(N / **N**d) for smoothing effect.
    - Suppose N=1,000,000,
    - Word **A** appears in 1 doc → N/df=1,000,000, Word **B** appears in 10 docs → N/df=100,000
    - Without log: ratio = **10x** difference. With log: log(1,000,000) ≈ 13.8, log(100,000) ≈ 11.5 Ratio ≈ **1.2x difference** → smoother, more realistic.
- TF-IDF: the product of TF and IDF gives more weight to distinctive terms.

Example:

Suppose we want to check similarity between:

•String A: "Data Mining Techniques"
•String B: "Database Mining Methods"

Tokenize and Build Vocabulary

Vocabulary across both strings:
{data, mining, techniques, database, methods}

Compute Term Frequency (TF)

For String A ("Data Mining Techniques"):
• data: 1
• mining: 1
• techniques: 1
• database: 0
• methods: 0

For String B ("Database Mining Methods"):
• data: 0
• mining: 1
• techniques: 0
• database: 1
• methods: 1

Compute Inverse Document Frequency (IDF)

Number of strings = 2.

- data → appears in 1 string → IDF = log(2/1) = 0.693
- mining → appears in 2 strings → IDF = log(2/2) = 0
- techniques → appears in 1 string → IDF = log(2/1) = 0.693
- database → appears in 1 string → IDF = 0.693
- methods → appears in 1 string → IDF = 0.693

Compute TF–IDF Vectors

String A:

- data = 1 * 0.693 = 0.693
- mining = 1 * 0 = 0
- techniques = 1 * 0.693 = 0.693
- database = 0
- methods = 0

→ Vector A = [0.693, 0, 0.693, 0, 0]

String B:

- data = 0
- mining = 1 * 0 = 0
- techniques = 0
- database = 1 * 0.693 = 0.693
- methods = 1 * 0.693 = 0.693

→ Vector B = [0, 0, 0, 0.693, 0.693]

Compute Cosine Similarity

Cosine similarity = $A \cdot B / ||A|| \cdot ||B||$

- Dot product = 0 (since no overlapping non-zero terms after IDF scaling).
- Norm of A = $\sqrt{(0.693^2 + 0.693^2)} \approx 0.98$
- Norm of B = $\sqrt{(0.693^2 + 0.693^2)} \approx 0.98$

Cosine similarity = 0 / (0.98*0.98) = **0.0**

- TF–IDF works best when distinguishing documents by **rare terms**.
- In practice, combining TF–IDF with **edit distance** or **Jaccard similarity** works better.

TF/IDF Similarity Score

- Let p and q be two strings, and T be the set of all terms in the collection
- Feature vectors $\mathbf{v}_p$ and $\mathbf{v}_q$ are vectors in the |T|-dimensional space wher each dimension corresponds to a term
- TF/IDF score of p and q is the cosine of the angle between $\mathbf{v}_p$ and $\mathbf{v}_q$
  - $s(p,q) = \sum_{t \in T} v_p(t) * v_q(t) \; / [\sqrt{\sum_{t \in T} v_p(t)^2} * \sqrt{\sum_{t \in T} vq(t)^2}]$

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering, bound filtering

Similarity based -- Ratcliff-Obershelp similarity

1. Find the longest common substring from the two strings.
2. Remove that part from both strings, and split at the same location. This breaks the strings into two parts, one left and another to the right of the found common substring.
3. Now take the left part of both strings and call the function again to find the longest common substring.
4. Repeat this for the right part.
5. Repeat Steps 1 to 4 recursively until the size of any broken part is less than a default value.

The similarity score = 2*the number of characters found in common/ the total number of characters in the two strings

Example

- S1 = M A T H E M A T I C S
- S2 = M A T E M A T I C A

S1: MKUESH MOHANIA
S2: MUKHESH MOHANIYA

- Step 1: Common substring - **E M A T I C**
  S1 = M A T H E M A T I C S
  S2 = M A T E M A T I C A

Step 2: Left String 1: M A T H Left String 2: M A T
Right String 1: S Right String 2: A

Step 3: Common substring in left string: **M A T**
S1 = M A T H E M A T I C S
S2 = M A T E M A T I C A

Step 4: None
Step 5: Algorithm terminates

Similarity score = 2*9/(11+10) = 0.857

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering, bound filtering

Phonetic Similarity Measures

- Match strings based on their sound, instead of appearances
- Very effective in matching names, which often appear in different ways that sound the same
  - e.g., Banarjee, Benerji, Banergi;
  - Meyer, Meier, Myir , Mier, and Mire;
  - Smith, Smithe, and Smythe;
  - Vohara, Vhora, Vohra, Vora
- Soundex is most commonly used

The Soundex Measure

- Used primarily to match surnames
  - maps a surname x into a 4-letter code
  - two surnames are judged similar if share the same code
- Algorithm to map x into a code:
  - Step 1: keep the first letter of x, subsequent steps are performed on the rest of x
  - Step 2: remove all occurences of W and H. Replace the remaining letters with digits as follows:
    - replace B, F, P, V with 1, C, G, J, K, Q, S, X, Z with 2, D, T with 3, L with 4, M, N with 5, R with 6
  - Step 3: replace sequence of identical digits by the digit itself
  - Step 4: Drop all non-digit letters, return the first four letters as the Soundex code
- Soundex code is padded with 0 if there is not enough digits

x = Banarjee
y= Benerji

Step 2: x= Ba5a62ee
y=Be5e62i

Step 3: : x= Ba5a62ee
y=Be5e62i

Step 4: : x= B562
y=B562

Home Assignment (1 mark)

1. Select any dataset from
https://sites.google.com/site/anhaidgroup/useful-stuff/the-magellan-data-repository

2. Apply different combinations of string matching methods to ANY ONE matching attribute.

3. Identify which combination of methods gives the overall maximum score to chosen matching attribute.

4. **Last Date of completion: October 12 till 9:30AM**

5. **You will be asked to present the findings in the class on Oct 12.**

Outline

- Problem description
- Matching methods and accuracy challenges
- Similarity measures
  - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex

- **Scaling up string matching**
  - Inverted index, size filtering, prefix filtering, position filtering

Computing similarity between two strings/record/documents -- Naïve Solution

Problem: Given large collection(s) of strings, efficiently find pairs that are similar under a chosen similarity/distance (Edit Distance, Overlap, Jaccard, Cosine, etc.).

- All pair-wise comparison between *A* and *B*

```
For x in A:
        For y in B:
            If sim(x,y) > t, return (x,y);
```

A, B: table
x, y: record as set

- Nested-loop: $|A||B|$ comparisons
- The *sim()* evaluation may be costly

Naïve Solution Example

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

***O(x,y)* > 2 ?**

O(x,y)	ID=4	ID=5	ID=6
ID=1	1	0	2
ID=2	2	2	**3**
ID=3	2	1	**3**

Naïve Solution Example

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

***J(x,y)* > 0.6 ?**

J(x,y))	ID=4	ID=5	ID=6
ID=1	0.25	0	0.5
ID=2	0.5	0.4	**0.75**
ID=3	0.2	0.16	0.6

Scalability Challenges

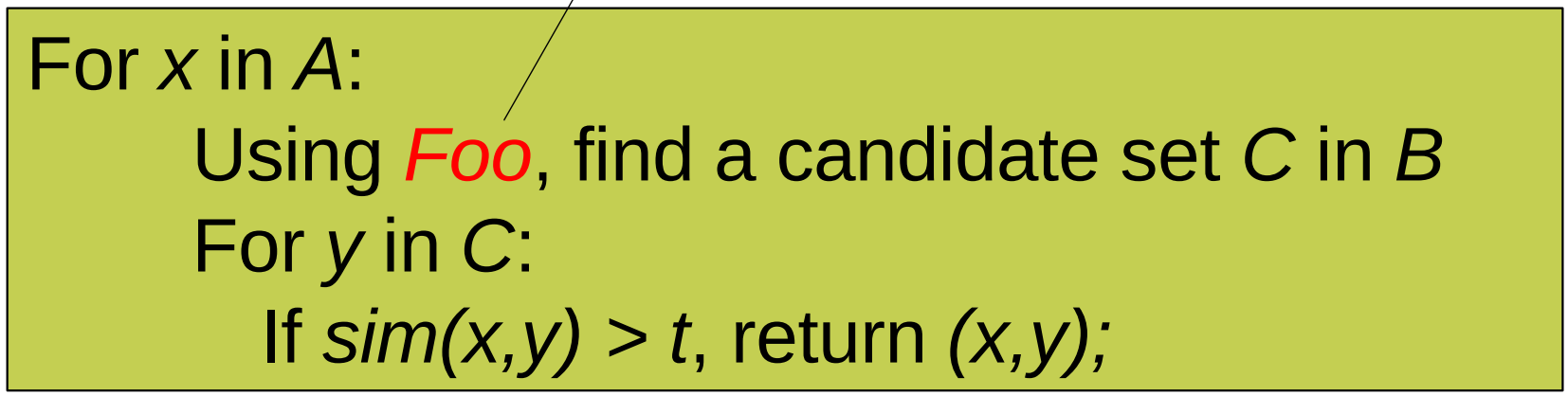
- Applying s(x,y) to all pairs is impractical
  - Quadratic in size of data. Naïve $O(n^2)$ comparison infeasible for millions of strings
- Solution: apply s(x,y) to only most promising pairs, using a method FindCands
  - For each string x ∈ X
    - use method ***FindCands*** to find a candidate set Z ⊂ Y for each string y ∈ Z
      - if s(x,y) ≥ t then return (x,y) as a matched pair
  - This is often called a blocking solution
- We now discuss ways to design/implement ***FindCands***
  - using Jaccard and overlap measures for now

2-Step Framework

High-level strategy (filter & verify):

1. Filter: cheaply eliminate most non-candidates.
2. Verify: run an exact (but expensive) similarity/distance test on remaining candidates.

- Step 1: "Blocking"
  - Using Index/heuristics/filtering/etc, reduce # of candidates to compare
- Step 2: sim() only within candidate sets



- Let S be a set of strings. For a string s of length $|s|$:

  character-level q-grams: $Gq(s) = [g1, g2, \dots, g|s|-q+1]$, where $|Gq(s)| = |s| - q + 1$

- Edit distance (Levenshtein) between s, t:

  $ED(s, t)$. Q-gram overlap: $ovq(s, t) = |Gq(s) \cap Gq(t)|$.

  - Each edit operation can affect at most q q-grams. So if $ED(s, t) \le k$, *then at most* $(q \cdot k)$ –grams can be "affected".

    For example: In "Vishwan…" → "Vishvan…", **Replace w → v**

    Bigrams involving **w**: ("hw", "wa"), They break, replaced by ("hv", "vn").

    ✓ Affects 2 bigrams (≤ q=2).

- Key inequality (q-gram overlap lower bound for edit distance):

  If $ED(s, t) \le k$ then

$$ovq(s, t) \ge \max(|Gq(s)|, |Gq(t)|) - q \cdot k$$

Let's take: s= "vishwanathan", t= "vishvnahtan", q=2, k=4.

$|G2(s)|$=11, $|G2(t)|$=10

RHS = max(11,10)−2·4=11−8=3, So the inequality holds since LHS=5 and RHS =3.

Variants for "*Foo*"

- *"Foo"*: How to identify candidate set *C*
  - Fast
  - Accurate: no false positives/negatives
- Many Variants for "Foo"
  - Inverted Index
  - Size filtering
  - Prefix Index
  - Prefix + Inverted Index
  - Position Index

Inverted index (core structure)

Idea: Tokenize strings (q-grams or words), build inverted lists mapping token -> list of string IDs and positions.

Construction (q-grams): For each string s, for each q-gram $g \in Gq(s)$ append entry (id(s), pos) to inverted list $\boldsymbol{I}[g]$.

Use at query/join time: For a query string q, iterate its tokens, probe the inverted lists to collect candidate string IDs with counts of shared tokens (overlap). Use counts + other filters to prune.

Complexity:

Index size ≈ sum of $|Gq(s)|$ across strings.

Inverted Index over Strings [Sarawagi et al, SIGMOD 04]

- Convert each string y∈Y into a document, build an inverted index over these documents
- Given term t, use the index to quickly find documents of Y that contain t

Inverted Index [Sarawagi et al, SIGMOD 04]

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

Inverted Index (IDX) for A

Token in A	ID List
Area	2
Dane	3
Lake	1, 2, 3
Mendota	1, 3
Monona	2, 3

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

Inverted Index (IDX) for B

Token in B	ID List
Area	5, 6
Lake	4, 6
Mendota	6
Monona	4, 5, 6
Research	5
University	4

Inverted Index [Sarawagi et al, SIGMOD 04]

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

For *x* in *A*:
Using *IDX*, find a candidate set *C* in *B*
For *y* in *C*:
If *sim(x,y)* > *t*, return *(x,y);*

Inverted Index (IDX) for B

Token in B	ID List
Area	5
Lake	4, 6
Mendota	6
Monona	4, 5, 6
Research	5
University	4

ID=1: {Lake, Mendota}

ID=2: …

ID=3: …

Candidate set C:
{4,6} + {6} = {4, 6}

Inverted Index [Sarawagi et al, SIGMOD 04]

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

For *x* in *A*:
  Using *IDX*, find a candidate set *C* in *B*
  For *y* in *C*:
    If *sim(x,y)* > *t*, return *(x,y);*

Inverted Index (IDX) for B

Token in B	ID List
Area	5
Lake	4, 6
Mendota	6
Monona	4, 5, 6
[illegible]	5
[illegible]	4

ID=1: {Lake, Mendota}

ID=2: …

ID=3: …

Candidate set C:

ID	Freq.
4	✗
6	✗

O(x,y) > 2

Limitations

- The inverted list of some terms (e.g., stop words) can be very long ➔ costly to build and manipulate such lists
- Requires enumerating all pairs of strings that share at least one term. This set can still be very large in practice.

Size Filtering [Arasu et al,VLDB 06]

- Retrieves only strings in Y whose sizes make them match candidates
  - given a string x∈X, infer a constraint on the size of strings in Y that can possibly match x
  - uses a B-tree index to retrieve only strings that satisfy size constraints
- For example, Edit distance $\leq k \Rightarrow \big|\,|x| - |y|\,\big| \leq k$.
- For Jaccard measure $J(x,y) = |x \cap y| / |x \cup y|$
  - assume two strings x and y match if $J(x,y) \geq t$
  - To match x with threshold t, the length |y| must satisfy: $|x|/t \geq |y| \geq |x|*t$ can possibly match x

- Usage: Precompute length buckets. When processing a string, only compare with strings whose length is within allowed bounds.
- Complexity effect: Reduces candidate universe drastically when lengths vary.

Example

Set X

1: {lake, mendota}
2: {lake, monona, area}
3: {lake, mendota, monona, dane}

Set Y

4: {lake, monona, university}
5: {monona, research, area}
6: {lake, mendota, monona, area}

(a)

Terms in Y	ID Lists
area	5 , 6
lake	4, 6
mendota	6
monona	4, 5, 6
research	5
university	4

(b)

- Consider x = {lake, mendota}. Suppose t = 0.8
- If y∈Y matches x, we must have
  - $2/0.8 = 2.5 \geq |y| \geq 2 * 0.8 = 1.6$
  - no string in Set Y satisfies this constraint ➔ no match

Prefix Filtering [Chaudhuri et al, ICDE 06]

- Key idea: if two sets share many terms ➔ large subsets of them also share terms
- Consider overlap measure $O(x,y) = |x \cap y|$
  - if $|x \cap y| \geq k$ ➔ any subset $x' \cdot x$ of size at least $|x| - (k - 1)$ must overlap y
- For J(x,y), any subset $x' \cdot x$ of size at least $|x| - (k - 1)$ must overlap y,

  where $k = \lceil |x| * t \rceil$
- To exploit this idea to find pairs (x,y) such that $O(x,y) \geq k$
  - given x, construct subset x' of size $|x| - (k - 1)$
  - use an inverted index to find all y that overlap x'

Example

x: {lake, monona, area}
x'

y: {lake, mendota, monona, area}

(a)

Set X

1: {lake, mendota}
2: {lake, monona, area}
3: {lake, mendota, monona, dane}

Set Y

4: {lake, monona, university}
5: {monona, research, area}
6: {lake, mendota, monona, area}
7: {dane, area, mendota}

(b)

Terms in Y	ID Lists
area	5, 6, 7
lake	4, 6
mendota	6, 7
monona	4, 5, 6
research	5
university	4
dane	7

(c)

- Consider matching using $O(x,y) \geq 2$
- **x**2 = {lake, monona, area}, let **x**2' = {lake, monona}
- Use inverted index to find {**y**4, **y**6} which overlaps with **x**2'.

Probing all subsets increases cost. Use an ordering and probe only prefixes.

Prefix-Inverted Index: Selecting the Subset Intelligently

[Bayardo et al,WWW 07]

- Recall that in Prefix Filtering we select a subset x' of x and check its overlap with the entire set y. Probing all subsets increases the cost.
- We can do better by selecting a particular subset x' and checking its overlap with only a particular subset y' of y.
- How?
  - impose an ordering O over the universe of all possible terms in increasing frequency. This makes prefixes more discriminative.
  - reorder the terms in each $x \in X$ and $y \in Y$ according to O
  - select subset x' that contains the first n terms of x as the prefix of size n of x
  - if $|x \cap y| \geq k$, then x' and y' must overlap, where x' is the prefix of size $|x| - (k-1)$ of x and y' is the prefix of size $|y| - (k-1)$ of y.

Selecting the Subset Intelligently

- Algorithm
  - reorder terms in each x ∈ X and y ∈ Y in increasing order of their frequencies
  - for each y ∈ Y, create y', the prefix of size |y| - (k – 1) of y
  - build an inverted index over all prefixes y'
  - for each x ∈ X, create x', the prefix of size |x| - (k – 1) of x, then use above index to find all y such that x' overlaps with y'

Prefix + Inverted Index [Bayardo et al,WWW 07]

A

ID	Content
1	{Lake, Mendota}
2	{Lake, Monona, Area}
3	{Lake, Mendota, Monona, Dane}

B

ID	Content
4	{Lake, Monona, University}
5	{Monona, Research, Area}
6	{Lake, Mendota, Monona, Area}

Inverted Index (IDX) for both A and B

Create a universal order: Put rare tokens front

Order: Dane > Research > University > Area > Mendota > Lake > Monona

Token	ID List	TF	Order
Area	2, 5, 6	3	4
Dane	3	1	1
Lake	1, 2, 3, 4, 6	5	6
Mendota	1, 3, 6	3	5
Monona	2, 3, 4, 5, 6	5	7
Research	5	1	2
University	4	1	3

Prefix + Inverted Index [Bayardo et al,WWW 07]

Ordered A

ID	Content
1	{Mendota, Lake}
2	{Area, Lake, Monona}
3	{Dane, Mendota, Lake, Monona}

Ordered B

ID	Content
4	{University, Lake, Monona}
5	{Research, Area, Monona}
6	{Area, Mendota, Lake, Monona}

Order: Dane > Research > University > Area > Mendota > Lake > Monona

Prefix + Inverted Index [Bayardo et al,WWW 07]

Ordered A

ID	Content
1	{Mendota, Lake}
2	{Area, Lake, Monona}
3	{Dane, Mendota, Lake, Monona}

Ordered B

ID	Content
4	{University, Lake, Monona}
5	{Research, Area, Monona}
6	{Area, Mendota, Lake, Monona}

$O(x,y) > 2$
Prefix(x)=|x|-(k-1)=|x|-1

Prefix Inverted Index for B

Token in B	ID List
Area	5, 6
Lake	4, 6
Mendota	6
Research	5
University	4

ID=1: {Mendota, Lake}

ID=2: …

ID=3: …

Candidate set C: {6}

Prefix + Inverted Index [Bayardo et al,WWW 07]

Ordered A

ID	Content
1	{Mendota, Lake}
2	{Area, Lake, Monona}
3	{Dane, Mendota, Lake, Monona}

Ordered B

ID	Content
4	{University, Lake, Monona}
5	{Research, Area, Monona}
6	{Area, Mendota, Lake, Monona}

$O(x,y) > 2$
Prefix(x)=|x|-(k-1)=|x|-1

Prefix Inverted Index for B

Token in B	ID List
Area	5
Lake	4, 6
Mendota	6
Research	5
University	4

ID=1: …

ID=2: {Area, Lake, Monona}

ID=3: …

Candidate set C:
{5} + {4,6} = {4,5,6}

Prefix + Inverted Index [Bayardo et al,WWW 07]

Ordered A

ID	Content
1	{Mendota, Lake}
2	{Area, Lake, Monona}
3	{Dane, Mendota, Lake, Monona}

Ordered B

ID	Content
4	{University, Lake, Monona}
5	{Research, Area, Monona}
6	{Area, Mendota, Lake, Monona}

$O(x,y) > 2$
Prefix(x)=|x|-(k-1)=|x|-1

Prefix Inverted Index for B

Token in B	ID List
Area	5
Lake	4, 6
Mendota	6
Research	5
University	4

ID=1: …

ID=2: …

ID=3: {Dane, Mendota, Lake, Monona}

Candidate set C:
{6} + {4,6} = {4,6}

if $|x \cap y| \geq k$, then x' and y' must overlap, where x' is the prefix of size $|x| - (k - 1)$ of x and y' is the prefix of size $|y| - (k - 1)$ of y.

Proof:

university < research
< dane < area
< mendota < monona < lake

(a)

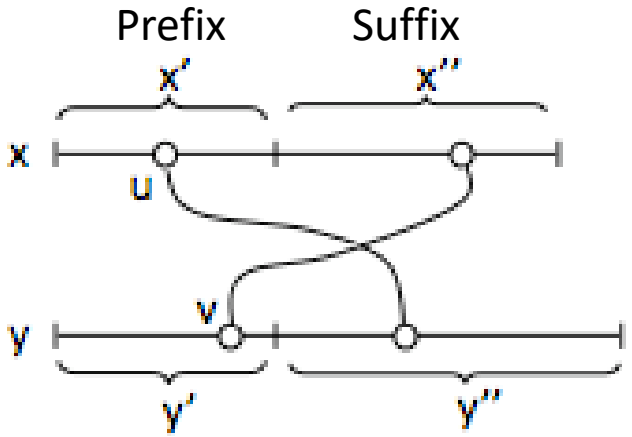
Reordered Set X

1: {mendota, lake}
2: {area, monona, lake}
3: {dane, mendota, monona, lake}

Reordered Set Y

4: {university, monona, lake}
5: {research, area, monona}
6: {area, mendota, monona, lake}
7: {dane, area, mendota}

(b)



(c)

Suppose $x' \cap y' = \Phi \cdot x' \cap y'' \neq \Phi$ (otherwise $|x \cap y| \geq k$ will not hold)
$\exists u$: $u \epsilon x'$ and $u \epsilon y''$ and similarly $\exists v$: $v \epsilon y'$ and $v \epsilon x''$
$u \epsilon x'$ and $v \epsilon x''$, $u < v$ in the ordering O
But $v \epsilon y'$ and $u \epsilon y''$, means $v < u$ -> contradiction
This x' overlaps with y'

Position Index [Xiao et al,WWW 08]

Further limits the set of candidate matches by deriving an upper bound on the size of overlap between x and y

Order: Dane > Research > University > Area > Mendota > Lake > Monona

- x: {Dane, Research, Area, Mendota, Lake}
- y: {Research, Area, Mendota, Lake, Monona}
- O(x,y) > 4 ?

➔

- Prefix(x) = Prefix(y) = 5 – (4 -1) = 2
- x: {Dane, Research, [redacted]}
- y: {Research, Area, [redacted]}
- "Research" is a common prefix ➔ (x,y) is a candidate pair using a Prefix Filtering + Inverted Index method
  - ➔ need to compute sim(x,y)
- Estimation of max overlap = overlap in prefixes + min of unseen tokens = 1 + min(3,3) = 4 > k ➔ No need to compute sim(x,y) !

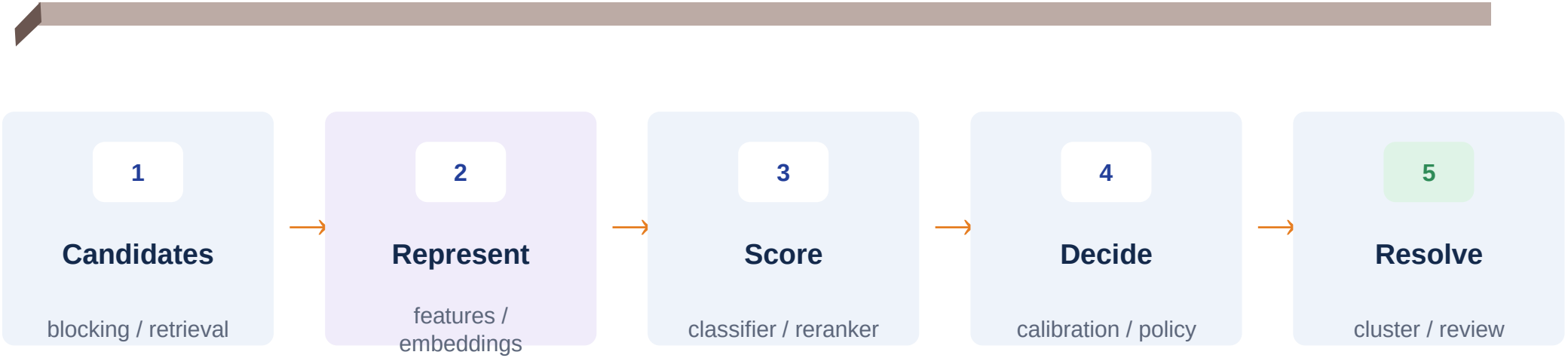
Classical Methods: Summary

- String matching is pervasive in data integration
- Two key challenges:
  - what similarity measure and how to scale up?
- Similarity measures
  - Sequence-based: edit distance, Needleman-Wunch, affine gap, Smith-Waterman, Jaro, Jaro-Winkler
  - Set-based: overlap, Jaccard, Sorensen-Dice, TF/IDF
  - Similarity-based: Ratcliff-Obershelp similarity
  - Phonetic: Soundex
- Scaling up string matching
  - Inverted index, size filtering, prefix filtering, position filtering.

Modern ML/DL Matching

Learned similarity, neural retrieval, reranking, calibration and scalable deployment

Where ML/DL fits in the matching pipeline



Classical measures remain useful—as features, retrieval signals and safeguards.

- ML learns how to combine evidence.
- DL learns representations and interactions.
- Systems engineering determines whether accuracy survives at scale.

Supervised matching as pair classification

Input pair

r_i = customer record
r_j = candidate record

→

Feature vector $\varphi(r_i, r_j)$

name JW • address TF-IDF • phone exact • date gap • missingness

→

Classifier

P(y=match | φ)
+ calibrated threshold

$$\hat{y} = 1[P(\text{match}) \geq \tau]$$

Why it improves accuracy

Weights and interactions are learned from labeled pairs instead of fixed manually.

Main limitation

Performance follows the label distribution; unseen domains and rare errors can fail.

Training pairs must reflect deployment

Positive pairs

verified links
transaction joins
reviewer-confirmed

Easy negatives

random pairs
use sparingly

Hard negatives

same block
similar name
different entity

Uncertain pairs

active-learning
review queue

The candidate generator changes the negative distribution.

- Split by entity, not by pair, to prevent identity leakage.
- Sample negatives from the same blocking process used in production.
- Track provenance: source, time window, annotator and ambiguity.

A feature vector combines heterogeneous evidence

Feature	Example	Value
Jaro–Winkler(name)	“M. Mohania” vs “Mukesh Mohania”	**0.88**
Jaccard(address tokens)	{12,mg,road} vs {12,m.g.,rd}	**0.50**
Exact(normalized phone)	+61…9123 vs +61…9123	**1**
|DOB difference|	0 days	**0**
Both missing(email)	false	**0**

$$\varphi = [0.88, 0.50, 1, 0, 0, \ldots]$$

Gradient-boosted trees are a strong tabular baseline

Strengths

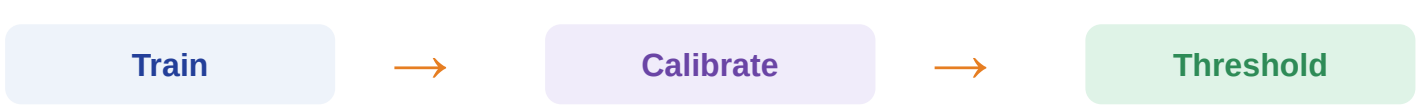
Non-linear interactions
Handles mixed features
Fast CPU inference
Feature attribution

Typical interaction

Name≈0.90 is weak alone, but strong with exact DOB and rare postcode agreement.

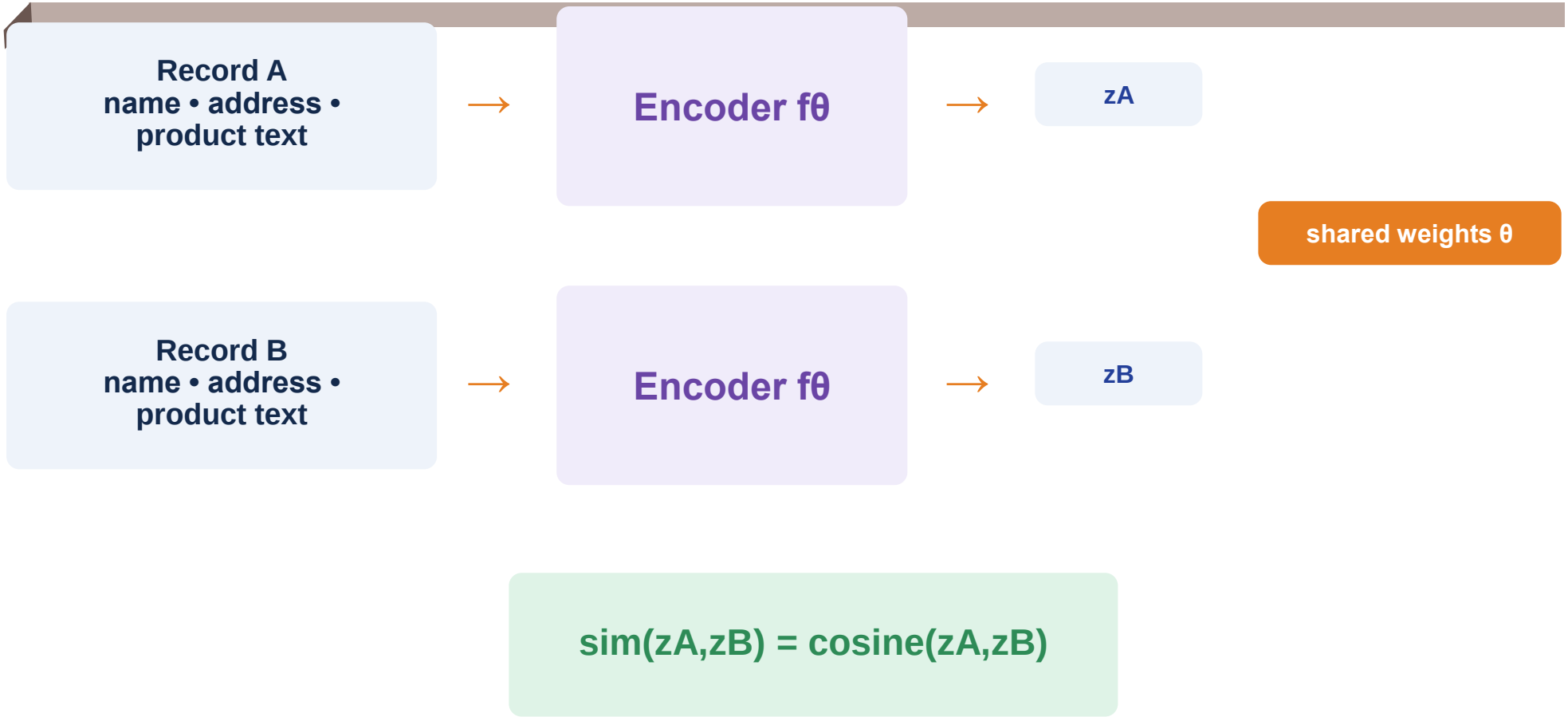
Risks

Poor probability calibration
Overfit source-specific artifacts
Biased labels
Candidate shift



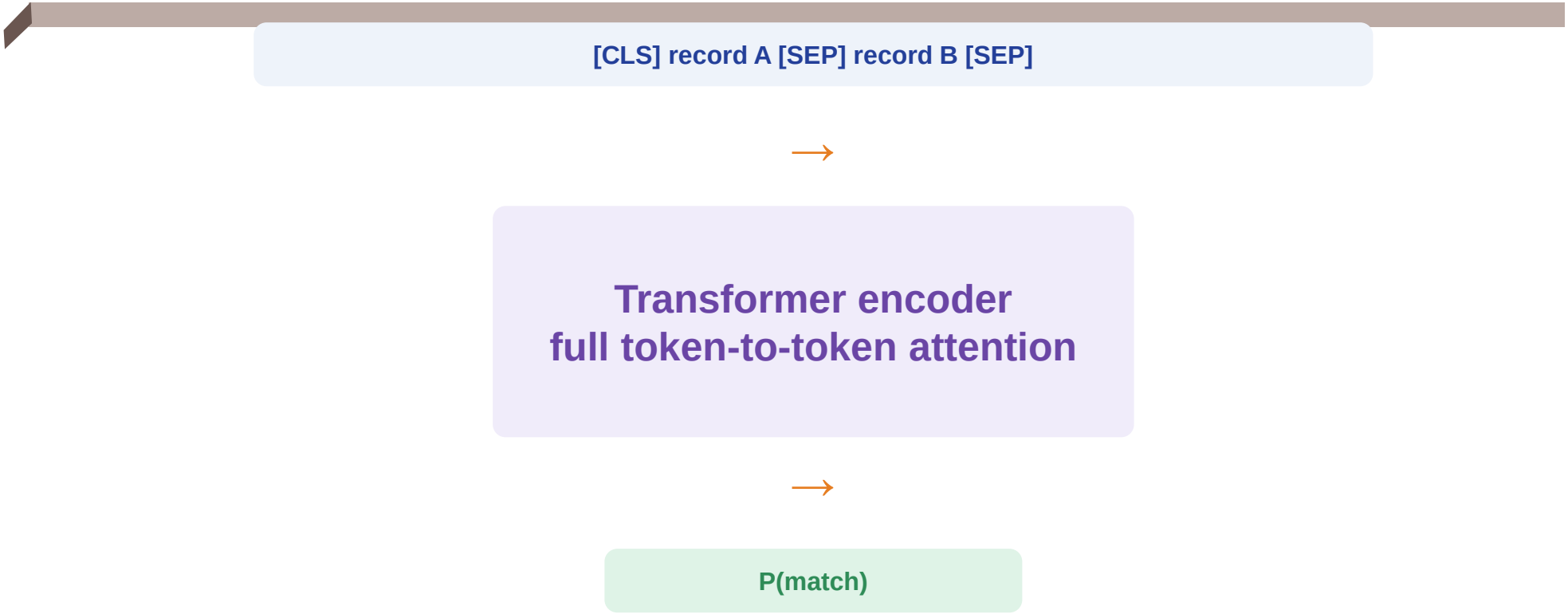
Do not choose τ from F_1 alone: include false-merge cost.

Siamese encoders learn a shared representation



Fast retrieval: encode each record once, then reuse its vector.

Cross-encoders model fine-grained interactions



Higher accuracy

Captures aligned and conflicting attributes jointly.

Higher cost

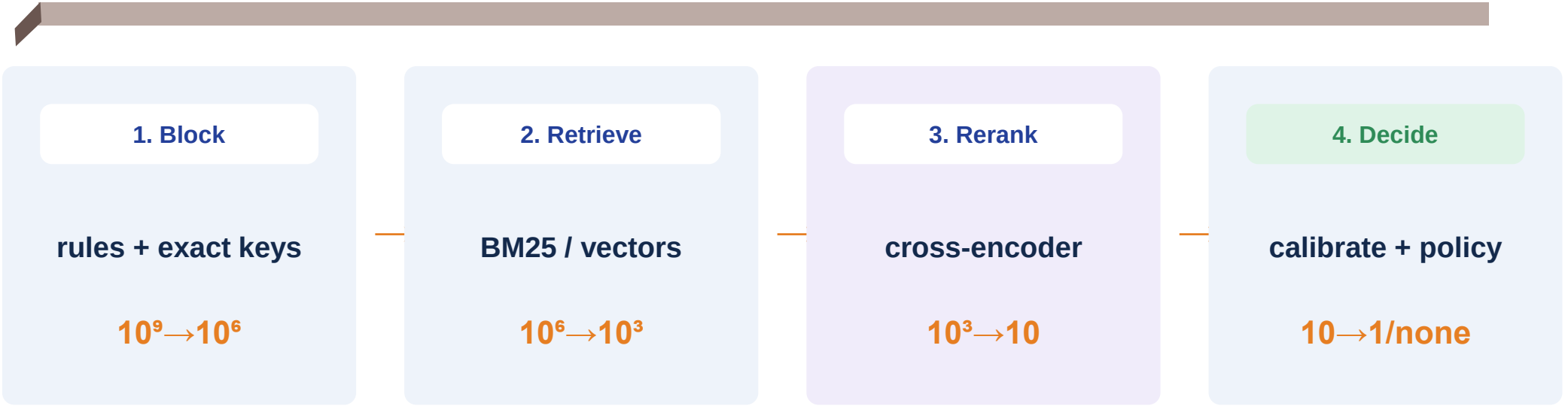
Must run once per pair; unsuitable for all-pairs search.

Bi-encoder versus cross-encoder

Property	Bi-encoder	Cross-encoder
Encoding	independent	joint pair
Retrieval	ANN-friendly	not directly
Pair interaction	limited	rich attention
Latency	low after indexing	higher per pair
Best role	candidate generation	reranking / final score

Best practice: retrieve cheaply, rerank selectively.

Retrieve–rerank architecture scales accuracy



Compute expensive models only where they can change the decision.

- Union candidates from multiple retrieval channels.
- Deduplicate candidates before reranking.
- Reserve a review band around the operating threshold.

Contrastive learning shapes the embedding space

Positive pair

pull embeddings together

Negative pair

push embeddings apart

Triplet loss

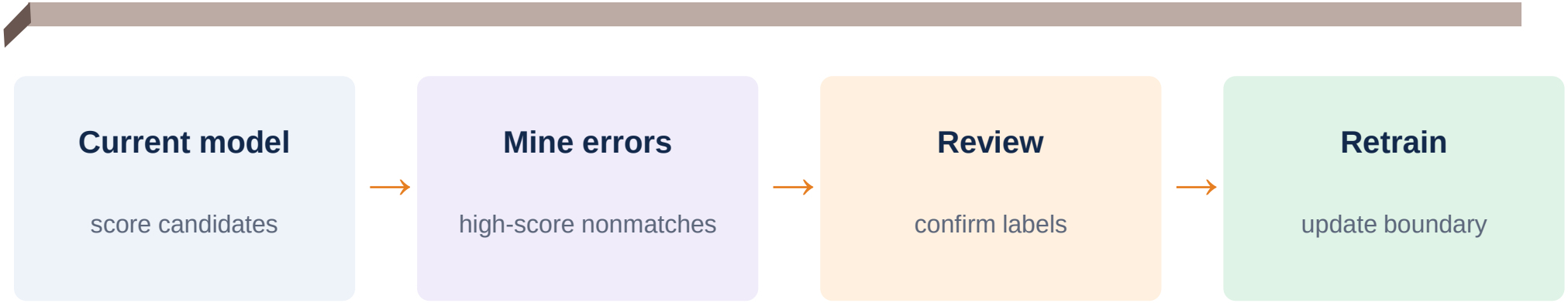
$$L = \max(0, d(a,p) - d(a,n) + \text{margin})$$

anchor a • positive p • negative n

Hard negatives create the useful gradient

Too-easy negatives teach separation that production blocking already solved.

Hard-negative mining improves discriminative power



Examples: same DOB + similar name, but different phone; same product model family, different capacity.

Guardrail

Mine from a time-separated pool and cap near-duplicates, otherwise training becomes narrow and unstable.

Approximate nearest-neighbor search

HNSW

graph traversal
high recall / memory

IVF

coarse partitions
fast large-scale search

PQ

compressed vectors
lower memory

LSH

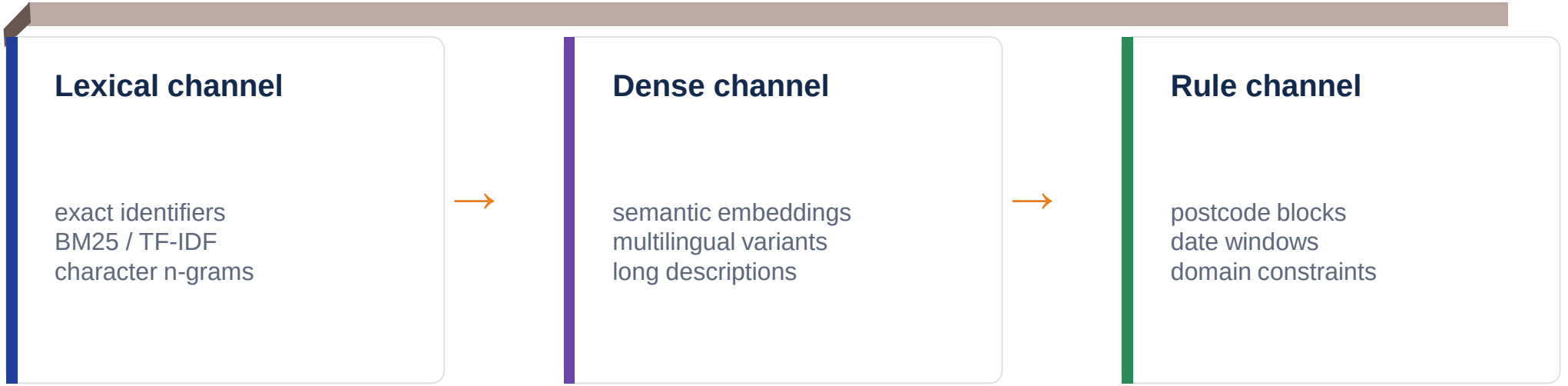
hash collisions
theoretical locality

Tune with retrieval recall@k, latency, memory and update rate—not ANN recall alone.



higher k → more pair completeness, more reranking cost

Hybrid retrieval combines complementary signals



Fusion options: union, weighted score, reciprocal-rank fusion, or learned ranker.

Calibration converts scores into decisions

Raw model score ≠ probability

AUTO-MERGE	REVIEW	REJECT
$p \geq 0.98$	$0.70 \leq p < 0.98$	$p < 0.70$

Calibrate

Platt scaling or isotonic regression on a representative held-out set.

Choose thresholds

Minimize expected cost under review capacity and reversibility.

Thresholds are business policy built on statistical evidence.

Evaluate retrieval, scoring and clustering separately

Retrieval

pair completeness
recall@k
reduction ratio

Pair scoring

precision / recall
PR-AUC
calibration

Clustering

B^3 / pairwise F_1
split / merge errors
cluster purity

Operations

latency / cost
review rate
drift / fairness

A high pairwise F_1 can still produce damaging entity clusters.

- Use entity-disjoint and time-based test sets.
- Report performance by source, language, geography and name frequency.
- Evaluate at the deployed candidate distribution and thresholds.

Prevent leakage and distribution shift

Pair leakage

same entity appears in train and test

Temporal leakage

future attributes inform past decisions

Block leakage

test negatives are easier than deployed candidates

Source leakage

model memorizes source-specific formatting

Entity-disjoint • time-sliced • block-consistent evaluation

LLMs are assistants, not the matching authority

Useful	Unsafe
Parse attributes from free text	Invent missing evidence
Normalize abbreviations with context	Return an uncalibrated yes/no
Generate candidate explanations	See unnecessary personal data
Create weak labels for review	Replace deterministic constraints

Recommended: LLM extraction → structured features → calibrated matcher → auditable decision.

Privacy

Minimize fields, redact identifiers, log prompts/versions and retain provenance.

Production architecture for accurate matching



Observability — recall proxy • drift • calibration • latency

Governance — provenance • versioning • unmerge • access control

Accuracy is an end-to-end property of the pipeline.

Design exercise: 50 million product records

Goal: match supplier catalog items to a master catalog with 99.5% pair completeness.

1 Candidates

Choose two retrieval channels and justify their failure complementarity.

2 Model

Choose GBDT, bi-encoder or cross-encoder for each pipeline stage.

3 Negatives

Construct hard negatives for model family, capacity and pack-size confusion.

4 Metrics

Specify retrieval, pair, cluster and latency metrics.

Constraint: ≤ 200 candidates/query and p95 latency ≤ 300 ms.

Modern matching: key takeaways

1	**Keep classical similarities**	They remain interpretable, cheap and valuable.
2	**Learn representations and weights**	Use ML/DL where rules cannot capture interactions.
3	**Separate retrieval from decision**	Optimize pair completeness before final precision.
4	**Calibrate and govern**	Thresholds, review and unmerge make scores operational.
5	**Measure the whole system**	Accuracy, scale, drift, fairness and cost are coupled.